FixIt – Adapting the Project to Cloud & Internet Computing and Software Engineering II

Nov 10, 2026 · @lda

Overview: one project, three courses

FixIt can cover all three courses because each one looks at the same system from a different angle. Web Development II builds the app. Cloud & Internet Computing deploys it on AWS with a public and a private network. Software Engineering II models its business logic with Domain-Driven Design (DDD).

Course	What FixIt gives that course	Main deliverable	Deadline
Software Engineering II (CEN3011)	The domain: tickets, triage, assignment, technicians, locations	ZIP with code, UML diagrams and a 3–5 page report; 10-minute presentation	Nov 29
Cloud & Internet Computing	The application deployed: S3 landing page, API server in a public VPC, database server in a private VPC	Progress report (items A–D)	Nov 24
		Word report (items E–G) + PowerPoint (15–20 slides) + demo video (5+ min)	Dec 1
		Final defense	Dec 3
Web Development II (COP 3851)	The full application (see the FixIt specification PDF)	Working app + presentation	Dec 7

Build order that serves all three: design the domain first (SE II), build the back end on that design (Web Dev), then deploy it (Cloud). The DDD model becomes the `domain/` folder of the Web Dev back end, and the AWS deployment runs that same back end.

Confirm these with your instructors this week

- [ ] **SE II, domain:** FixIt (facility maintenance) is not on the list of domains, so it needs approval. The guideline allows "propose your own with instructor approval."
- [ ] **SE II, language:** the guideline says Java or Python is *preferred*. Ask whether JavaScript/TypeScript is accepted, so the DDD code can be the real FixIt back end. If not, build the DDD prototype in Python (Part B explains both options).
- [ ] **Cloud, group:** the group table in the guideline does not list your name. Confirm your group or partner with Dr. Arias.
- [ ] **Cloud, scope:** confirm that deploying FixIt is acceptable. Step 2 already asks you to bring in what you learned in web development courses, so this should be an easy yes.
- [ ] **Timing conflict:** Cloud step 10 says to delete all AWS resources when you finish, but the Web Dev presentation is Dec 7, after the Cloud defense on Dec 3. Either keep the AWS resources running until Dec 7 (and watch the cost) or demo Web Dev from Docker on your laptop.

Part A – Cloud & Internet Computing: FixIt on AWS

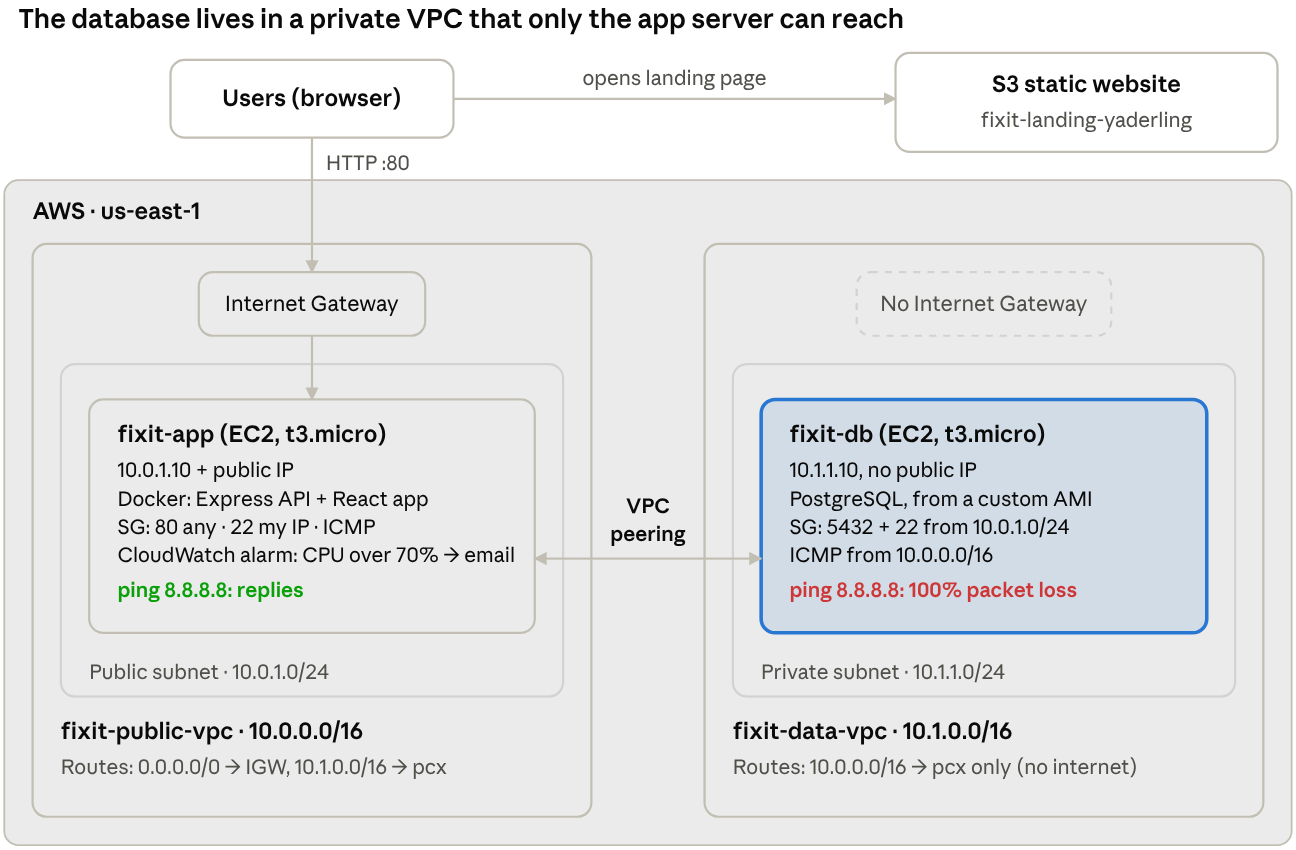
The guideline asks for the same AWS resources from every student: S3 static site, two VPCs, peering, two VMs, ping tests, an alarm and a cost estimate. FixIt gives each resource a real job. This is a common three-tier design: a public web/app tier and a private data tier that the internet cannot reach.

Step	Required by the guideline	FixIt version
1	MFA on the root account	Same. Also create an IAM admin user for daily work (security good practice to mention in the report).
2	S3 static website with a unique index.html	**FixIt landing page**: what FixIt is, the problem it solves, the architecture diagram, "Built by Yaderling Garcia", what you learned in Cloud and Web Dev, and a "Report a problem" button that links to the app on the public VM.

Step	Required by the guideline	FixIt version
3	VPC with internet + VPC without internet, no CIDR overlap, peering	**fixit-public-vpc** (10.0.0.0/16) for the app; **fixit-data-vpc** (10.1.0.0/16) for the database. Peering lets the API reach the database.
4	One VM in each VPC + network diagram	**fixit-app** (public): Docker running the Express API and the React build. **fixit-db** (private): PostgreSQL (and MongoDB if you keep it). Diagram in Part A architecture below.
5	Allow ICMP; ping 8.8.8.8 from both VMs	fixit-app reaches 8.8.8.8; fixit-db gets 100% packet loss. Extra proof: fixit-db **can** ping fixit-app through the peering.
6	Alert or alarm on one VM	CloudWatch alarm on fixit-app: CPU above 70% for 5 minutes, sends an email through SNS.
7	One-year cost with the pricing calculator	AWS Pricing Calculator: 2 EC2 instances 24/7, their EBS disks, the public IPv4 address, S3 storage, CloudWatch alarm.
8	Document everything (portfolio style)	Screenshots + diagram + a GitHub README with the deployment steps.
9	Practice the presentation (groups)	Present it as a job-interview walkthrough: problem, architecture, live demo, cost, lessons.
10	Delete all resources	Checklist in the last section. Mind the Web Dev demo date (see the Overview).

Part A – Architecture, setup and deliverables

Network diagram



FixIt AWS network · 2 VPCs, peering, 2 VMs, S3 site

Users reach FixIt only through the Internet Gateway of the public VPC; the database answers only to the app subnet across the peering. Redraw this in app.diagrams.net with the official AWS icons for your submission (the guideline asks for a drawing app).

Resource configuration

Resource	Name	Configuration
Region	—	us-east-1 (one region for everything; peering stays simple)
VPC (public)	fixit-public-vpc	10.0.0.0/16, subnet 10.0.1.0/24, Internet Gateway fixit-igw

Resource	Name	Configuration
Route table (public)	rt-public	0.0.0.0/0 → fixit-igw · 10.1.0.0/16 → fixit-peering
VPC (private)	fixit-data-vpc	10.1.0.0/16, subnet 10.1.1.0/24, **no** Internet Gateway, **no** NAT Gateway
Route table (private)	rt-private	10.0.0.0/16 → fixit-peering (no default route)
Peering connection	fixit-peering	Requester fixit-public-vpc, accepter fixit-data-vpc
VM (public)	fixit-app	t3.micro (or t2.micro), Amazon Linux 2023, private IP 10.0.1.10 + public IP, Docker
VM (private)	fixit-db	t3.micro, Amazon Linux 2023, private IP 10.1.1.10, no public IP, PostgreSQL
S3 bucket	fixit-landing-yaderling	Static website hosting, public read on objects only
Alarm	fixit-app-high-cpu	CPUUtilization > 70% for 5 min → SNS topic fixit-alerts → your email

Security group rules (the "firewall rules" for the diagram)

Security group	Inbound rule	Source	Why
sg-fixit-app	HTTP 80	0.0.0.0/0	Users open the FixIt app
sg-fixit-app	SSH 22	Your IP only	Administration
sg-fixit-app	ICMP (all)	0.0.0.0/0	Ping tests (step 5)
sg-fixit-db	PostgreSQL 5432	10.0.1.0/24	Only the API subnet can reach the database
sg-fixit-db	SSH 22	10.0.1.0/24	Reach the DB VM through fixit-app as a jump host
sg-fixit-db	ICMP (all)	10.0.0.0/16	Ping across the peering

Outbound stays at the default (all traffic). For fixit-db that has no effect on the internet, because its route table has no path out. Add the MongoDB port (27017) the same way if you keep MongoDB.

Deployment steps

1. Turn on root MFA, create an IAM admin user, and set an **AWS Budget alert** (for example $5) so you get an email before any surprise charges.
2. Create the S3 bucket, enable static website hosting, allow public read with a bucket policy, and upload `index.html`, a CSS file and the diagram image.
3. Create both VPCs, their subnets, the Internet Gateway (public VPC only) and both route tables.
4. Create the peering connection, accept it, then add the peer route to **both** route tables. Peering without routes does nothing, which is a common mistake worth mentioning in the report.
5. Launch fixit-app in the public subnet. Install Docker and Docker Compose, then clone the FixIt repository.
6. **Prepare fixit-db (the tricky step).** A VM with no internet cannot download PostgreSQL. Solution: launch a temporary VM in the public VPC, install and configure PostgreSQL (listen on its private IP, allow 10.0.1.0/24 in `pg_hba.conf`), create an **AMI**

from it, delete the temporary VM, and launch fixit-db from that AMI in the private subnet. This also produces the "image file" that step 8 suggests exporting.

7. Connect to fixit-db through fixit-app: `ssh -J ec2-user@<app-public-ip> ec2-user@10.1.1.10`.
8. On fixit-app, set `DATABASE_URL=postgresql://fixit:<password>@10.1.1.10:5432/fixit` in `.env`, run the Prisma migrations, start the containers, and open `http://<app-public-ip>`.
9. Add the ICMP rules and run the four ping tests: app → 8.8.8.8 (works), db → 8.8.8.8 (fails), db → app and app → db (work through the peering).
10. Create the SNS topic, confirm the email subscription, and create the CloudWatch alarm.
11. Build the one-year estimate in the AWS Pricing Calculator and export it.

Screenshots to collect: MFA enabled · S3 website open in a browser · peering connection (Active) · both route tables · both security groups · both instances running · the four ping outputs · the alarm configuration · the pricing estimate · FixIt running from the public IP.

Item C – Literature survey topics

Keep it short (about 1–2 pages), one paragraph per topic, each with a real source:

- **Cloud service models (IaaS, PaaS, SaaS):** NIST SP 800-145, *The NIST Definition of Cloud Computing*. FixIt on EC2 is IaaS; S3 website hosting is closer to PaaS.
- **Shared responsibility model:** AWS documentation. AWS secures the hardware; you secure security groups, passwords and patches.
- **Network segmentation and the three-tier architecture:** why databases sit in private networks.
- **VPC peering vs. other ways to connect networks:** AWS VPC Peering Guide (mention Transit Gateway as the larger-scale option).
- **Observability and cost optimization:** AWS Well-Architected Framework, the Operational Excellence and Cost Optimization pillars.

Item D – Improvement (choose one or two)

Improvement	What it adds	Effort
Ticket photos in S3 with an IAM role (recommended)	FixIt photo uploads go to a private S3 bucket. The fixit-app VM gets permission through an IAM instance role, so no access keys are stored on the server. Connects directly to the Web Dev upload feature.	Medium
CloudWatch agent for logs and memory	The API's Pino logs and memory usage appear in CloudWatch, with a small dashboard.	Low–Medium
S3 gateway VPC endpoint	Lets the private VM reach S3 (for example, backups to a bucket) without internet access. Gateway endpoints for S3 have no hourly charge.	Low
Database backup to S3	A cron job runs `pg_dump` on fixit-db and copies the file to S3 through the endpoint above.	Medium

Item E – Code to explain

`index.html` of the landing page, the S3 bucket policy (JSON), the `docker-compose.yml` and `.env` of FixIt, the security group rules, and the PostgreSQL settings (`postgresql.conf`, `pg_hba.conf`). Explain each in a short paragraph: what it does and why it is configured that way.

Item F – Word report outline

Title · Table of contents · Introduction (the problem and why a cloud deployment) · Procedures (the steps above with screenshots) · Results (ping outputs, working app, alarm, cost estimate) · Conclusions (what you learned, what you would change for production) · References.

Item G – PowerPoint (15–20 slides) and demo video (5+ minutes)

1. Title

2. The problem
3. What FixIt does
4. Cloud service models used
5. Architecture diagram
6. Public VPC
7. Private VPC
8. Peering and route tables
9. Security groups
10. S3 landing page
11. The two VMs
12. Ping test results
13. CloudWatch alarm
14. Improvement (item D)
15. One-year cost
16. Challenges (the no-internet database VM)
17. Lessons learned
18. Cleanup
19. Q&A

Video script: open the S3 landing page → click through to FixIt → report a problem with a photo → show the ticket stored in the private database → run the four pings live → show the alarm and the cost estimate.

Part B – Software Engineering II: FixIt with Domain-Driven Design

Proposed domain: campus facility maintenance (issue reporting, triage and work orders). It is not on the list, so ask for approval. Pitch it as close to the "Medical Appointment Scheduling" option in complexity, with a clearer real-world problem.

4.1 Domain analysis – interviews

Interview three people, real if possible (someone from your university's maintenance office would be ideal), or simulated. Label simulated interviews as simulated in the report.

Interviewee	Questions to ask	What you want to learn
Facilities manager	How do problems reach you today? How do you decide who fixes what? What do you need to report to your boss?	Assignment rules, priorities, metrics
Technician	What information do you need before you go? When is a job "done"? What makes a job urgent?	Status lifecycle, the meaning of "resolved", hazards
Student or employee	How do you report a problem now? Do you ever hear back? Would you try a fix yourself if guided?	Reporting pain points, self-help acceptance

Key business processes: report an issue → triage it → self-resolve *or* escalate to a work order → dispatch to a technician → work → resolve → requester confirms and rates → close.

Business rules found (examples to confirm in the interviews):

- A hazard (gas, sparks, exposed wires, flooding, structural damage) is always Urgent and never gets self-help steps.
- A work order can be dispatched only to a technician who has the matching trade (electrical, plumbing, HVAC, IT/AV).
- Only the assigned technician can start or resolve a work order.
- A work order follows a fixed lifecycle; it cannot jump from Open to Resolved.
- The requester can cancel only before dispatch, and can rate only after resolution.
- An Urgent work order not dispatched within 1 hour, or any work order open more than 48 hours, breaks its SLA.

Subdomains

Type	Subdomain	Why
Core	Issue Triage & Work Order Management	What makes FixIt valuable: fast, safe triage and reliable dispatch. Most design effort goes here.

Type	Subdomain	Why
Supporting	Facility Directory	Buildings, rooms and categories. Needed, but simple CRUD.
Supporting	Workforce	Technicians, their trades and current workload.
Supporting	Maintenance Analytics	Resolution times, recurring locations, self-resolution rate.
Generic	Identity & Access	Login and roles. Solved problem (JWT, bcrypt).
Generic	Notifications	Sending messages. Any library or service works.
Generic	Image analysis (AI)	Bought from an external AI provider, not built.

4.2 Bounded contexts

The guideline asks for at least two. Implement the first two in code; describe the rest in the context map.

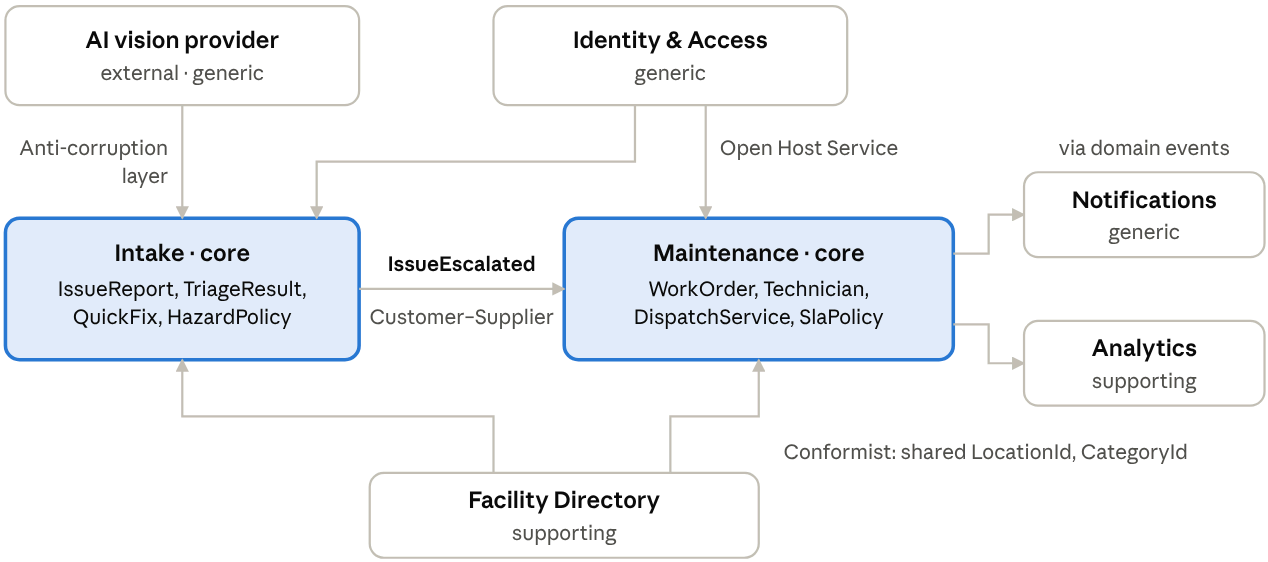
Bounded context	Responsibility	Inside the boundary	Outside the boundary
Intake (implemented)	Receive an issue, triage it, offer safe quick fixes, decide whether to escalate	IssueReport, TriageResult, QuickFix, HazardPolicy	Who fixes it, how long it takes
Maintenance (implemented)	Manage work orders from creation to closure	WorkOrder, Assignment, Technician, SLA, Rating	How the issue was described or analyzed
Facility Directory	Keep the list of buildings, rooms and categories	Building, Room, Category	Any work order logic
Identity & Access	Authenticate users and give them roles	Account, Role, credentials	Any maintenance concept

Bounded context	Responsibility	Inside the boundary	Outside the boundary
Notifications	Tell people when something changes	Message, channel, recipient	Business rules

The same thing has different names in different contexts. This is a key DDD point for the report: in Intake, the person is a **Reporter** who files an **Issue Report** (which might be fixed without any ticket). In Maintenance, the same person is a **Requester** and the escalated report becomes a **Work Order**. In Identity, they are just an **Account**. Each context keeps its own model instead of sharing one big "User" or "Ticket" class.

Context map

Intake hands work to Maintenance through one event; the AI sits behind a translation layer



FixIt context map · 2 core contexts, 5 surrounding

Arrows point from upstream to downstream. The two highlighted contexts are the ones you implement in code.

Upstream → Downstream	Relationship pattern	How they integrate
Intake → Maintenance	Customer–Supplier + **Published Language**	Intake publishes an `IssueEscalated` event; Maintenance creates a Work Order from it.
AI provider → Intake	**Anti-Corruption Layer**	A `TriageGateway` translates the AI's JSON into the domain's `TriageResult` value object, so AI changes never leak into the model.
Identity & Access → all contexts	**Open Host Service**	Exposes `accountId` + role (JWT claims); each context maps them to its own terms (Reporter, Requester, Technician).
Facility Directory → Intake, Maintenance	**Conformist**	Both use the Directory's `LocationId` and `CategoryId` as they are.
Maintenance → Notifications, Analytics	Published Language (domain events)	`WorkOrderDispatched`, `WorkOrderStatusChanged`, `WorkOrderResolved` are consumed downstream.

Part B – Model, implementation and report

4.3 Ubiquitous language (glossary)

Use exactly these words in the diagrams, class names, method names, the README and the presentation.

Term	Context	Meaning	In code
Issue Report	Intake	A problem someone describes, with a photo and a location. It may be solved without a work order.	`IssueReport`

Term	Context	Meaning	In code
Reporter	Intake	The person who files an Issue Report.	`ReporterId`
Triage	Intake	Classifying an issue: category, priority, hazard, possible quick fix.	`triage()`
Triage Result	Intake	The outcome of triage. Cannot change once created.	`TriageResult`
Quick Fix	Intake	Safe self-help steps a non-technician can follow.	`QuickFix`
Hazard	Intake, Maintenance	A dangerous situation (gas, sparks, exposed wires, flooding, structural damage).	`is_hazard`, `HazardPolicy`
Self-Resolved	Intake	The reporter fixed it with the Quick Fix; no Work Order is created.	`mark_self_resolved()`
Escalate	Intake	Send the issue to Maintenance as a Work Order.	`escalate()` → `IssueEscalated`
Work Order	Maintenance	A job for a technician, from creation to closure.	`WorkOrder`
Requester	Maintenance	The person waiting for the Work Order to be done.	`RequesterId`
Technician	Maintenance	Staff member with one or more Trades who does the work.	`Technician`
Trade	Maintenance	A skill area: Electrical, Plumbing, HVAC, IT/AV, General.	`Trade`
Dispatch	Maintenance	Assign a Work Order to a qualified Technician.	`dispatch()`

Term	Context	Meaning	In code
Resolution	Maintenance	What the technician did to fix it.	`Resolution`
SLA	Maintenance	The time limit for dispatch and resolution, set by priority.	`SlaPolicy`
Location	Facility Directory	Building + floor + room.	`Location`

4.4 Domain model

Aggregates, entities and value objects

Building block	Context	Element	Notes
Aggregate root	Intake	`IssueReport`	Holds its `TriageResult`; decides self-resolve vs. escalate
Aggregate root	Maintenance	`WorkOrder`	Contains `TimelineEntry` entities and an optional `Resolution` and `Rating`
Aggregate root	Maintenance	`Technician`	Trades and active workload; referenced from WorkOrder by `TechnicianId` only
Entity	Maintenance	`TimelineEntry`	Has its own id; only changed through `WorkOrder`

Building block	Context	Element	Notes
Value objects	Shared kernel / both	`Priority`, `Category`, `Location`, `Photo`, `WorkOrderStatus`, `Trade`, `Rating` (1–5), `SlaDeadline`, `TriageResult`, `QuickFix`, IDs	Immutable, compared by value, validate themselves
Domain services	Intake	`HazardPolicy`	Forces Urgent + removes quick fixes when hazard keywords appear, even if the AI says otherwise
Domain services	Maintenance	`DispatchService`	Picks the qualified technician with the lowest workload (needs data from two aggregates, so it does not belong to either)
Domain services	Maintenance	`SlaPolicy`	Computes deadlines from priority; checks for breaches
Repositories	Both	`IssueReportRepository`, `WorkOrderRepository`, `TechnicianRepository`	Interfaces in the domain; SQLite/JSON (SE II) or Prisma (Web Dev) implementations in infrastructure
Domain events	Both	`IssueEscalated`, `WorkOrderDispatched`, `WorkOrderStatusChanged`, `WorkOrderResolved`, `SlaBreached`	Connect contexts and drive notifications

Invariants (rules the aggregate root always protects)

- `WorkOrder` status moves only along: Open → Dispatched → In Progress → Resolved → Closed; Cancelled only from Open.
- Only the dispatched technician can call `start()` and `resolve()`.

- `resolve()` requires a non-empty Resolution; `rate()` works only when Resolved and moves the order to Closed.
- A hazard Work Order always has priority Urgent.
- `IssueReport` can be self-resolved only if its TriageResult has a QuickFix and no hazard.
- Every state change adds exactly one TimelineEntry and raises one domain event.

Lifecycle: draw a UML **state diagram** for `WorkOrder` (the six states above) in addition to the class diagram. It is the clearest picture of the invariants.

PlantUML starter (class diagram)

```
@startuml
package "Intake" {
  class IssueReport <<AggregateRoot>> {
    id: IssueReportId
    reporterId: ReporterId
    description: String
    photo: Photo
    location: Location
    triage(gateway, hazardPolicy)
    markSelfResolved()
    escalate(): IssueEscalated
  }
  class TriageResult <<ValueObject>> {
    category: Category
    priority: Priority
    isHazard: bool
    quickFix: QuickFix [0..1]
  }
  interface IssueReportRepository
  class HazardPolicy <<DomainService>>
  IssueReport *-- TriageResult
}
package "Maintenance" {
  class WorkOrder <<AggregateRoot>> {
    id: WorkOrderId
    requesterId: RequesterId
    status: WorkOrderStatus
    priority: Priority
    technicianId: TechnicianId [0..1]
    dispatch(technician)
```

```
    start(by)
    resolve(by, resolution)
    rate(rating)
    cancel(by)
  }
  class TimelineEntry <<Entity>>
  class Resolution <<ValueObject>>
  class Technician <<AggregateRoot>> {
    trades: Set<Trade>
    activeWorkload: int
    canHandle(category): bool
  }
  class DispatchService <<DomainService>>
  class SlaPolicy <<DomainService>>
  interface WorkOrderRepository
  WorkOrder *-- "many" TimelineEntry
  WorkOrder *-- "0..1" Resolution
  WorkOrder ..> Technician : by id only
  DispatchService ..> Technician
  DispatchService ..> WorkOrder
}
IssueReport ..> WorkOrder : IssueEscalated event
@enduml
```

5 Implementation

Language decision (ask the instructor):

Option	What you build	Pros	Cons
A. TypeScript/JavaScript (if approved)	The DDD layers *are* the FixIt back end: `domain/`, `application/`, `infrastructure/` inside the Web Dev project	One codebase for two courses; Express routes call application services	Needs approval, since Java/Python is "preferred"

Option	What you build	Pros	Cons
B. Python (safe default)	A standalone DDD prototype of Intake + Maintenance, with SQLite persistence and a small CLI	Matches the guideline exactly; small and focused	A second codebase, but the model is the same one the Web Dev back end follows

Folder structure (Option B, Python)

```
fixit-ddd/
├── fixit/
│   ├── shared/            # value objects used by both contexts (Priority,
Category, Location, IDs)
│   ├── intake/
│   │   ├── domain/        # IssueReport, TriageResult, QuickFix,
HazardPolicy, events, repository interface
│   │   ├── application/   # ReportIssue, ConfirmSelfResolved, EscalateIssue
(use cases)
│   │   └── infrastructure/# SQLite repository, FakeTriageGateway (the anti-
corruption layer)
│   └── maintenance/
│       ├── domain/        # WorkOrder, TimelineEntry, Technician,
DispatchService, SlaPolicy, events
│       ├── application/   # CreateWorkOrder (handles IssueEscalated),
Dispatch, Start, Resolve, Rate
│       └── infrastructure/# SQLite repositories, in-memory event bus
├── cli.py                 # demo: report → triage → escalate → dispatch →
resolve → rate
├── tests/                 # pytest: invariants and policies
├── docs/                  # PlantUML files + exported diagrams
└── README.md              # how each DDD pattern is applied, with file links
```

How each required pattern shows up in the code

Required pattern	Where	How to show it
Entities with identity and behavior	`WorkOrder`, `TimelineEntry`, `Technician`	Equality by id; methods like `dispatch()` instead of setters

Required pattern	Where	How to show it
Value objects with immutability	`Priority`, `Location`, `TriageResult`, `Rating`	`@dataclass(frozen=True)`; validation in `__post_init__` (e.g., Rating 1–5)
Aggregates with root enforcement	`WorkOrder`	Timeline entries are added only by WorkOrder methods; no public list to append to; invalid transitions raise `DomainError`
Repositories	`WorkOrderRepository` (abstract base class) + `SqliteWorkOrderRepository`	Domain code never imports sqlite3
Domain services	`DispatchService`, `HazardPolicy`, `SlaPolicy`	Stateless; work across aggregates or encode policy

Example: aggregate root protecting an invariant

```
class WorkOrder:
    _ALLOWED = {
        Status.OPEN: {Status.DISPATCHED, Status.CANCELLED},
        Status.DISPATCHED: {Status.IN_PROGRESS},
        Status.IN_PROGRESS: {Status.RESOLVED},
        Status.RESOLVED: {Status.CLOSED},
    }

    def start(self, by: TechnicianId) -> None:
        if by != self.technician_id:
            raise DomainError("Only the dispatched technician can start this
work order")
        self._move_to(Status.IN_PROGRESS, actor=by)

    def _move_to(self, new: Status, actor) -> None:
        if new not in self._ALLOWED.get(self.status, set()):
            raise DomainError(f"Cannot go from {self.status.value} to
{new.value}")
        old, self.status = self.status, new
        self._timeline.append(TimelineEntry.status_changed(old, new, actor))
        self._events.append(WorkOrderStatusChanged(self.id, old, new))
```

Optional enhancements (both are easy wins): SQLite persistence and the CLI demo above. Under Option A, the Express API and React UI from Web Dev already cover "a simple web interface."

Link to Web Dev: the Web Dev spec's status rules, `hazardCheck()`, the dispatch logic and the Socket.io events are exactly `WorkOrder`'s invariants, `HazardPolicy`, `DispatchService` and the domain events. Mention this in both presentations.

6 Report (3–5 pages) and presentation (10 minutes)

Report outline:

1. The domain and why it matters (half a page)
2. Design decisions and how DDD shaped them: two contexts instead of one "Ticket" model, the anti-corruption layer for AI, the aggregate boundaries, Technician referenced by id
3. Challenges and how you solved them (for example: where dispatch belongs, why it became a domain service; deciding the aggregate size)
4. Reflection on the value of DDD

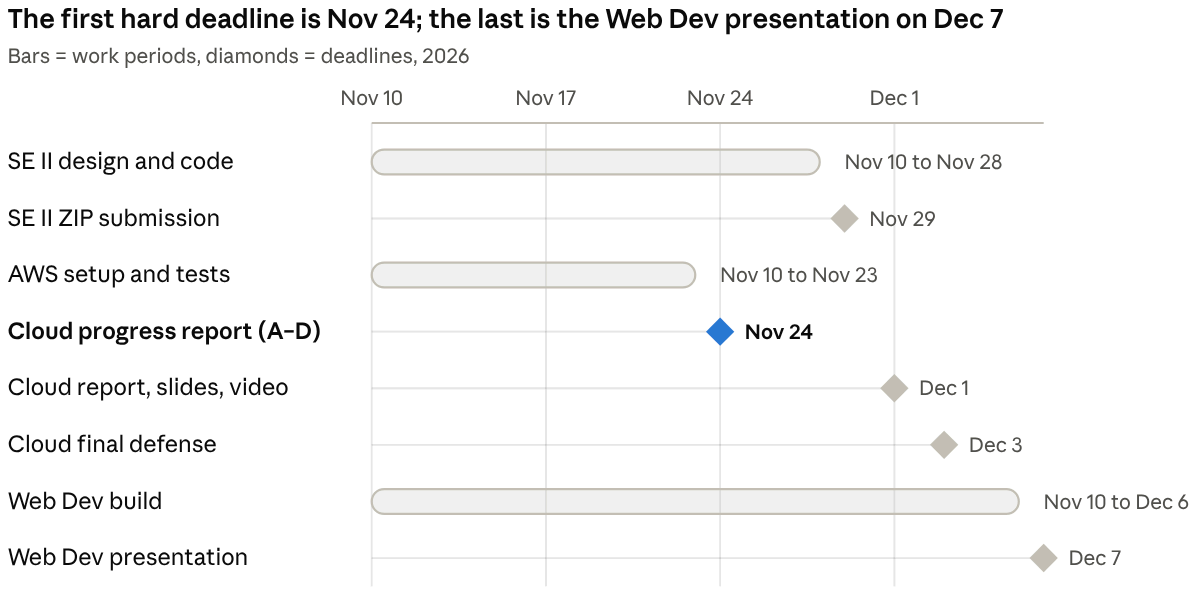
Presentation (10 minutes):

Time	Content
1 min	Domain and problem
2 min	Subdomains and context map
1 min	Ubiquitous language: Reporter vs. Requester, Issue Report vs. Work Order
3 min	Domain model + live demo of an invariant (a refused invalid transition)
2 min	Code walkthrough: aggregate root, value object, repository, domain service
1 min	What DDD clarified + Q&A

Grading weights to keep in mind: code quality and DDD application 40% · domain modeling 30% · documentation and reflection 20% · presentation 10%.

Combined timeline and checklist

The order is SE II design first, Cloud infrastructure in parallel, and Web Dev features on top. The first hard deadline is the Cloud progress report on **Nov 24**.



Deadlines across the three courses · Nov 10 to Dec 7

Week by week

Week	Software Engineering II	Cloud & Internet Computing	Web Development II
Nov 10–16	Get domain + language approved; interviews; subdomains, contexts, glossary; context map and class diagram	MFA, IAM user, budget alert; S3 landing page; literature survey draft	Repo, Docker Compose, Prisma schema, auth + roles
Nov 17–23	Implement Intake + Maintenance domain, tests, SQLite + CLI	VPCs, peering, both VMs, pings, alarm, pricing; write and submit **progress report (A–D) by Nov 24**	Work order CRUD, status rules, filters, Socket.io

Week	Software Engineering II	Cloud & Internet Computing	Web Development II
Nov 24–30	README, report, slides; **submit ZIP by Nov 29**	Deploy FixIt on AWS; improvement (item D); Word report, PowerPoint, demo video; **submit by Dec 1**	AI triage, dashboard, tests
Dec 1–7	—	**Final defense Dec 3**	Security review, Swagger, final Docker; **presentation Dec 7**; then delete the AWS resources

Keep the three versions consistent

- [ ] Use the SE II glossary in the Web Dev code too. For example, rename the Web Dev status `ASSIGNED` to `DISPATCHED`, and use "Work Order" in the UI if you adopt it in SE II.
- [ ] Reuse one architecture diagram style across the Cloud report and the Web Dev slides.
- [ ] Keep the screenshots from AWS before deleting anything; they are needed for the Cloud report and are useful for Web Dev too.

AWS cleanup checklist (Cloud step 10)

- [ ] Terminate fixit-app and fixit-db
- [ ] Deregister the fixit-db AMI and delete its snapshot
- [ ] Delete leftover EBS volumes and release any Elastic IP
- [ ] Delete the peering connection, then both VPCs (subnets, route tables, Internet Gateway go with them)
- [ ] Empty and delete both S3 buckets
- [ ] Delete the CloudWatch alarm and the SNS topic
- [ ] Check Billing → Bills and the Cost Explorer for anything still charging, in **every** region
- [ ] Sign out